

Date.....

* Java Servlets :- Servlets are Java classes that run on a server to handle HTTP requests and generate dynamic responses for web applications. Acts as a middle layer between clients and backend data sources.

* Why do we need Servlets?

Before Servlets, building web applications in pure Java was extremely difficult and manual.

→ Problem :-

- Manual HTTP handling → Without a container, a developer would have to write code to manually listen to a network port, parse incoming HTTP requests (extracting headers, body, parameters, endpoints), and manage complex multi-threading to handle multiple users simultaneously.
- Response Construction → After processing, you would also have to manually format an HTTP request/response with the correct headers & status codes.
- Focus Shifts → This manual work forces developers to focus on network-level plumbing.

rather than their actual business logic.

→ Solution:-

- A servlet container (like Tomcat) acts as a bridge. It manages the lifecycle of your Servlets, handles incoming requests, and routes them to correct code.
- Instead of raw sockets, you interact with `HttpServletRequest` and `HttpServletResponse` objects.
- The container handles mapping, so you just tell it: "When a request hits /hello, call the `hello()` method in my class".

* Project Setup :- To build a web application using Servlets, you need to structure your project to be compatible with a Servlet Container (like Apache Tomcat).

→ Embedded Tomcat :- Modern Spring Boot applications include Tomcat directly inside project, making it "run-and-go".

→ External Tomcat :- In traditional development, you download Tomcat server separately on your machine and deploy your compiled application onto it.

Date.....

→ WAR Files - While standalone JAVA apps use JAR (Java Archives) files, web applications use WAR (Web Application Archives).

A WAR file acts as a container for your web project, holding:

- Compiled Java class files (including your Servlets).
- Configuration files.
- Static web assets like HTML, CSS & Javascript.

→ Configurations:

pom.xml → You must set "packaging" tag to "war" in maven config.

Dependencies → You need to add the "jakarta.servlet-api" dependency to your project to use Servlet classes.

* Servlet Lifecycle :- Every Servlet goes through a specific sequence controlled by the container :-

- Initialization (init()) → Called once when the Servlet is first loaded. This is the place to perform heavy setup tasks, like loading expensive resources (similar to `postConstruct` in Spring).

- Service (service()) → The entry point for requests. By default, the `HttpServlet` implementation of this method inspects the HTTP method type (GET, POST, etc.) and routes it to appropriate `do...()` method.
- Destructive (destroy()) → Called before the Servlet is removed from memory, allowing you to perform cleanup tasks like closing connections or flushing caches.

→ Handling HTTP Methods:- When you override methods like `doGet()` or `doPost()`, you are providing the specific business logic for those actions.

- When Tomcat receives a request, it calls the `service()` method of your Servlet (or parent class if not overridden).
- The parent `HttpServlet` class then evaluates the request type (if (method == POST)) and invokes your specific `doPost()` implementation.
- This design ensures that you only focus on the logic for the specific request type.

→ Request and Response Objects :-

- HttpServletRequest → Contains all the incoming data (headers, query parameters, body). Tomcat parses the raw HTTP stream into this object for you.
- HttpServletResponse → A blank object provided by the container. You interact with it by setting status codes, content types, and writing the response body. Tomcat then converts this into a standard HTTP response to send back to the client.

* Path to MVC :-

→ Problem with Multiple Servlets :-

If you have dozens of different Servlets for different tasks, your code becomes scattered and difficult to manage. Every individual Servlet would need to handle its own routing, request parsing, and response formatting, leading to a lot of repetitive "boilerplate" code.

→ Centralized Approach

Instead of many independent Servlets, modern frameworks introduce a Central Servlet (in Spring, this is `DispatcherServlet`).

- This single point of entry receives all incoming requests.
- It acts as a "traffic controller", handling cross-cutting concerns like logging, validation, and security, before routing the requests to specific methods or components that handle that business logic.

→ Why Spring Boot over Embedded Tomcat?

- While we manually deployed to an external Tomcat earlier, Spring Boot bundles the Tomcat server inside your application.
- This ensures your application is self-contained and ready to run, abstracting away the complex configuration of external servers while still relying on the same robust Servlet foundation underneath.

*** Frameworks don't replace Servlet architecture; they sit on top of it, providing automated configuration and cleaner abstractions so you can focus entirely on your business requirements rather than network plumbing. ***